23CSE455 Design Patterns

Case Study Report

Team: B6

Case study: Vehicle rental and fleet management domain, ASP.NET Core framework: *Design Pattern Identification and Evolution in an ASP.NET Core-Based Vehicle Rental Application*

Application: WanderCar / Fleman Vehicle Rental and Fleet Management System, .NET backend (`https://github.com/Naveen-verma-7697/FleetManagment`).

Faculty approval date: 2026-10-02

Date: 2026-10-11

Submission repository: 23CSE455_DP_CaseStudy, folder `B6`

Students

Roll number	Student name	Batch
AM.SC.U4CSE23211	Bhavana P H	CSE C (2023–2027)
AM.SC.U4CSE23238	Neema Vinod	CSE C (2023–2027)
AM.SC.U4CSE23251	Samridhi Singh	CSE C (2023–2027)
AM.SC.U4CSE23266	Vani Sugovind S R	CSE C (2023–2027)

Contents

1 Application and architecture **3**

2 Part 1 Pattern discovery **4**

2.1 Totals at the baseline 4

2.2 Pattern inventory 5

2.3 Pattern instance records 5

2.3.1 Generic Repository 5

2.3.2 ASP.NET Core DI Composition 6

2.3.3 Booking Service Layer 7

2.3.4 ASP.NET Core Request Middleware Chain 7

2.3.5 Email Background Work Queue 8

2.3.6 Java Microservice Adapter 9

2.3.7 Legacy API Proxy 9

2.3.8 Legacy Java Integration 10

2.3.9 API Request/Response DTO Boundary 10

2.4 Rejected and uncertain candidates 11

3 Part 2 Evolution and refactoring **11**

3.1 CH01: Fleet-maintenance scheduling (State) 12

3.2 CH02: New vehicle categories (Factory Method) 15

3.3 CH03: Interchangeable pricing policies (Strategy) 18

3.4 CH04: Booking lifecycle notifications (Observer) 20
3.5 Integration and updated pattern counts 23

4 Part 3 Cross-language comparison **24**
4.1 Factory Method: Vehicle Category Rules Creation 25
4.2 Observer: Booking Lifecycle Notification 26
4.3 Dependency Injection: Service Registration and Constructor Injection 27
4.4 Strategy and Repository (smoke-level ports) 28

5 LLM records and human verification **29**

6 Findings, contributions and references **30**

7 Team folder and datasets **31**

1 Application and architecture

Original source: `https://github.com/Naveen-verma-7697/FleetManagment`, branch `main`. Baseline commit `dfc61ab8dd6f9bbb2301171457dc03946a249576` (2026-08-24, "Clean up comments in IAvailabilityRepository"). Accessed October 2026. The repository has no licence file, so no licence is stated. The baseline source is stored unchanged in `B6/data/code/WanderCar_DotNet`.

Environment:

Language and framework	C# 12, ASP.NET Core 8 Web API
Persistence	EF Core 8 with Pomelo MySQL provider; MySQL 8 to run the application
Main libraries	AutoMapper, FluentValidation, MailKit, QuestPDF
Tests	NUnit 3.14.0, NUnit3TestAdapter 4.5.0, Moq 4.20.72, FluentAssertions 6.12.1, EF Core InMemory 8.0.2 (no database needed)
Build machine	Windows 11 Pro 10.0.26300, .NET SDK 8.0.425, runtime 8.0.31
Diagrams	PlantUML 1.2024.8 (Smetana layout) and Graphviz `.dot` sources

Scope: Inspected: the .NET backend `WanderCar_DotNet/src/FlemanApi` (folders `Controllers`, `Service`, `Repository`, `Data`, `Models`, `DTO`, `Middleware`, `Security`, `Util`, `Validators`, `AutoMapper` and `Program.cs`) and its test project `FlemanApi.Tests`. This covers the eight functional areas of the brief: vehicle management, customer management, reservation/booking, pricing, payment/billing, maintenance, notification and authentication. The file-by-file list is `B6/Part1/inspected_modules.txt`. Excluded: the React frontend and the Java Spring Boot backend. Neither is part of the .NET application; the Java backend is reached only over HTTP through `IJavaMicroserviceClient`, which is in scope. EF Core migrations are generated code and are excluded from line counts.

Architecture: The backend is a layered Web API (Figure 1). HTTP requests pass a middleware chain (exception handling, request logging) into 13 thin controllers. Controllers delegate to services, which hold the business workflows. Services use a generic repository and an availability repository over `FlemanDbContext` (EF Core, MySQL), queue e-mail on a bounded background channel that a hosted service drains through MailKit, and reach the legacy Java backend through an HTTP client.

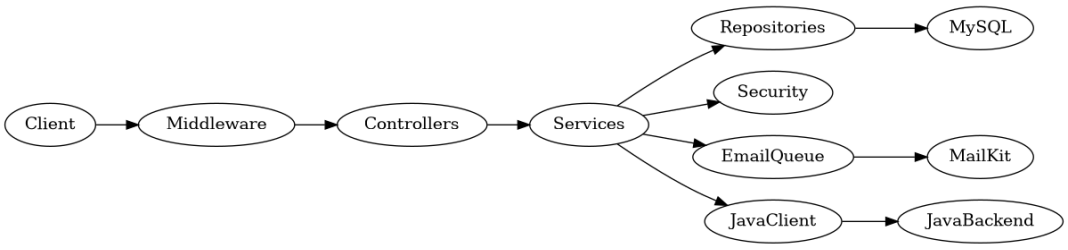


Figure 1: Component view of the original code (source: `Part1/pattern_architecture/architecture.dot`).

Request flow: creating a booking (original code).

1. `POST /api/bookings` passes `ExceptionHandlingMiddleware` and `RequestLoggingMiddleware`, each calling `_next(context)`.
2. `BookingController` calls `IBookingService.CreateBookingAsync` (`Controllers/BookingController.cs:30`).
3. `BookingService` validates the car type, dates and hubs, checks capacity through `IAvailabilityRepository`, and prices the booking with the static `RentalRateCalculator`.
4. The booking header and add-on lines are saved through `IGenericRepository<,>` and

FlemanDbContext.

5. The confirmation e-mail is built inline and handed to IEmailService, which enqueues it on EmailBackgroundQueue; EmailQueueHostedService later sends it through MailKitEmailSender.

Steps 3 and 5 are exactly the parts that Changes CH03 (pricing) and CH04 (notifications) later modify.

Run instructions: README: README.md at the repository root. Run from the folder that contains WanderCar_DotNet:

```
cd B6\data\code\WanderCar_DotNet
dotnet test src\FlemanApi.Tests\FlemanApi.Tests.csproj
```

The test project is named because DotnetBackend.slnx needs SDK 9.0.200 or later and fails on SDK 8 with MSB4068. If no NuGet source is configured, add -source https://api.nuget.org/v3/index.json to a first dotnet restore.

Actual result (2026-10-11): build 0 warnings, 0 errors; 183 tests, 182 passed, 1 failed (B6/data/test_result/). The failing test, StaffServiceTests.ProcessReturnAsync_CreatesInvoiceHeaderAndDetails, depends on the date it is run: the fixture hands the car over on 2026-08-01 while StaffService bills up to DateTime.Now, so the expected add-on amount 40.0 is only produced on 2026-08-05. It is a pre-existing defect, not a regression; CH03 later removes it by injecting a clock.

To run the application itself, start MySQL 8, set ConnectionStrings:Default in src/FlemanApi/appsettings.json and run dotnet run -project src/FlemanApi; Swagger is at /swagger in Development.

2 Part 1 Pattern discovery

Method:

1. **Scope fixed.** Baseline commit dfc61ab8; inspected files listed in Part1/inspected_modules.txt.
2. **Candidate generation.** The relevant source was extracted into Part1/llm/input/part1_source_context.txt and given to an LLM (GPT-5.6 Luna, 2026-10-08) with the prompt in Part1/Part1_prompt.txt. The raw answer is Part1/llm/output/pattern_discovery_raw.txt. It proposed 21 candidates.
3. **Source check.** Each candidate was traced in the baseline files: the interfaces and classes that play each role, the calls and delegation between them, and where they are wired in Program.cs. A class name, an annotation or the LLM's assertion alone was never accepted as evidence.
4. **Classification.** Each candidate was recorded as confirmed, uncertain or rejected with a written reason. Framework APIs that only resemble a pattern (for example AddSingleton or WebApplication.CreateBuilder) were not counted as application-authored patterns.

Line ranges below were taken from the baseline source in B6/data/code; a range covers the whole type declaration unless a single line is given.

2.1 Totals at the baseline

Confirmed pattern types: 9: Repository, Dependency Injection / IoC, Service Layer, Middleware Pipeline / Chain of Responsibility, Producer–Consumer, Adapter, Proxy, Strangler Fig,

Data Transfer Object.

Confirmed pattern instances: 9, one independent collaboration per type.

Concrete participant classes: 15 distinct classes in variable roles (17 counted per instance, because `BookingService` and `EmailBackgroundQueue` each appear in two instances): `GenericRepository`, `BookingService`, `VehicleService`, `EmailBackgroundQueue`, `ExceptionHandlingMiddleware`, `RequestLoggingMiddleware`, `EmailService`, `EmailQueueHostedService`, `MailKitEmailSender`, `JavaMicroserviceClient`, `LegacyProxyController`, `JavaInvoicePdfService`, `CreateBookingRequest`, `BookingResponseDTO`, `CarDTO`.

Unique participant symbols: 28 distinct classes, interfaces and modules across the nine participant tables, shared symbols counted once (framework types such as `RequestDelegate` and the external Java service are not counted).

Rejected and uncertain candidates: 9 rejected and 3 uncertain (Section 2.4). Observation scope: the baseline `FlemanApi` source only.

2.2 Pattern inventory

Pattern type (category)	Instance name	Instances	Concrete classes
Repository (enterprise)	Generic Repository	1	1
Dependency Injection / IoC (architectural)	ASP.NET Core DI Composition	1	3
Service Layer (enterprise)	Booking Service Layer	1	1
Middleware Pipeline / Chain of Responsibility (architectural, GoF behavioural)	ASP.NET Core Request Middleware Chain	1	2
Producer–Consumer (concurrency)	Email Background Work Queue	1	4
Adapter (GoF structural)	Java Microservice Adapter	1	1
Proxy (GoF structural)	Legacy API Proxy	1	1
Strangler Fig (modernisation)	Legacy Java Integration	1	1
DTO (enterprise)	API Request/Response DTO Boundary	1	3
Total	9 types	**9**	**17** (15 distinct)

Data files: `Part1/part1_patterns.csv`, `Part1/part1_pattern_count.csv`.

2.3 Pattern instance records

All paths are relative to `WanderCar_DotNet/src/FlemanApi/` at commit `dfc61ab8`. For every record the origin is the application (none is student-added), the verification status is **confirmed**, and the reviewer is Team B6.

2.3.1 Generic Repository

Pattern type and category: Repository (enterprise application pattern, Fowler).

Purpose and origin: Keeps EF Core query and persistence code out of the services: services depend on a CRUD interface instead of on `DbContext`. Application code.

Pattern role	Actual symbol	File and lines
Repository	`IGenericRepository<TEntity, TKey>`	`Repository/IGenericRepository.cs:6-17`
ConcreteRepository	`GenericRepository<TEntity, TKey>`	`Repository/GenericRepository.cs:11-52`
Persistence context	`FlemanDbContext`	`Data/FlemanDbContext.cs:6-49`
Client	`VehicleService`	`Service/VehicleService.cs:10-105`
Client	`BookingService`	`Service/BookingService.cs:17-459`
Registration	open-generic `AddScoped(typeof(IGenericRepository<,>), ...)`	`Program.cs:179`

Evidence: `IGenericRepository` declares the CRUD and query operations; `GenericRepository` implements them over `Context.Set<TEntity>()` and exposes `SaveChangesAsync`. Services receive typed repositories (e.g. `IGenericRepository<Car,long>`) and never touch `DbSet` directly. One open-generic registration serves every entity.

Diagram: Figure 2; source `Part1/pattern_architecture/repository.dot`.

Verification: Confirmed. Alternatives considered: Unit of Work (kept uncertain, see Section 2.4).

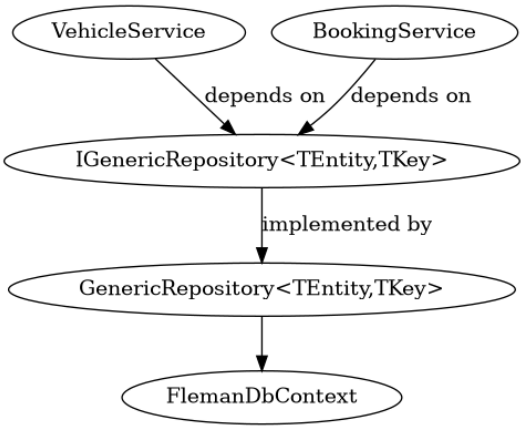


Figure 2: Generic Repository.

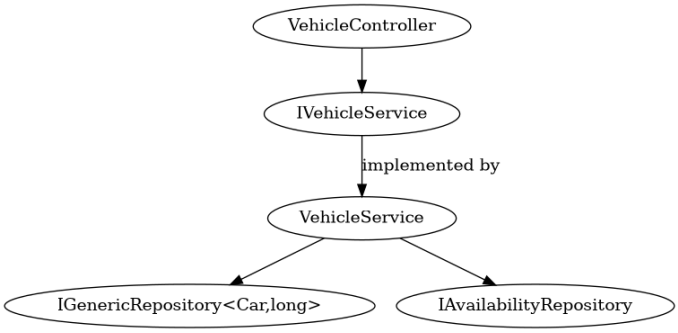


Figure 3: Service layer around `VehicleService`.

2.3.2 ASP.NET Core DI Composition

Pattern type and category: Dependency Injection / Inversion of Control (architectural; framework idiom).

Purpose and origin: Objects do not create their collaborators; a composition root binds abstractions to implementations and the container supplies them through constructors. The container is framework-provided; the bindings and constructor signatures are application code.

Pattern role	Actual symbol	File and lines
Composition root	service registrations in `Program.cs`	`Program.cs:150-204`
Abstraction → implementation	`IBookingService` → `BookingService`	`Program.cs:187`
Abstraction → implementation	`IVehicleService` → `VehicleService`	`Program.cs:186`
Abstraction → implementation	`IEmailBackgroundQueue` → `EmailBackgroundQueue` (singleton)	`Program.cs:190`
Constructor-injected client	`BookingController`	`Controllers/BookingController.cs:11-56`
Constructor-injected client	`VehicleController`	`Controllers/VehicleController.cs:10-31`
Injector	ASP.NET Core `IServiceProvider`	framework

Evidence: Every service and controller declares its dependencies as interface-typed constructor parameters; `Program.cs` registers interface/implementation pairs with scoped, singleton or hosted lifetimes, plus a typed `HttpClient` for the Java client (`Program.cs:204`).

Verification: Confirmed. `AddSingleton` is a container lifetime, so it was not counted as the GoF Singleton.

2.3.3 Booking Service Layer

Pattern type and category: Service Layer (enterprise application pattern, Fowler).

Purpose and origin: Defines the application's operations in one layer so controllers stay thin and the booking workflow is reusable and testable. Application code.

Pattern role	Actual symbol	File and lines
Remote facade / controller	`BookingController`; delegating calls	`Controllers/BookingController.cs:11-56` (calls at 30, 37, 41, 47, 51, 55)
Service interface	`IBookingService`	`Service/IBookingService.cs:5-13`
Service implementation	`BookingService`	`Service/BookingService.cs:17-459`
Domain/data dependencies	`IGenericRepository<,>`, `IAvailabilityRepository`	`Repository/IAvailabilityRepository.cs:8-21`

Evidence: Each `BookingController` action is a single call to `IBookingService`; validation, capacity checks, pricing, persistence and notification all live in `BookingService`. The vehicle area repeats the same structure (Figure 3); it is the same layering and is not counted as a second instance.

Verification: Confirmed.

2.3.4 ASP.NET Core Request Middleware Chain

Pattern type and category: Middleware Pipeline / Chain of Responsibility (architectural; GoF behavioural).

Purpose and origin: Cross-cutting request handling (error translation, logging) is applied as a chain of handlers, each deciding whether and when to pass the request on. Handler classes are application code; the pipeline mechanism is the framework's.

Pattern role	Actual symbol	File and lines
ConcreteHandler	`ExceptionHandlingMiddleware.InvokeAsync`	`Middleware/ExceptionHandlingMiddleware.cs:14-56` (`await _next(context)` at 33)
ConcreteHandler	`RequestLoggingMiddleware.InvokeAsync`	`Middleware/RequestLoggingMiddleware.cs:9-36` (`_next` at 26)
Successor	`RequestDelegate _next`	framework
Chain assembly	`app.UseMiddleware<...>()`	`Program.cs:220-221`

Evidence: Both handlers receive the next `RequestDelegate` in their constructor and invoke it inside their own logic; the exception handler wraps the call in `try/catch` and turns `ApiException` into an `ErrorResponse`. The registration order fixes the chain: exception handling outermost, then logging.

Diagram: Figure 4; source `Part1/pattern_architecture/middleware_chain.dot`.

Verification: Confirmed. Decorator was rejected for the same code (no shared component interface).



Figure 4: Request middleware chain.

2.3.5 Email Background Work Queue

Pattern type and category: Producer–Consumer (concurrency pattern).

Purpose and origin: Sending SMTP mail is slow and can fail; requests should not wait for it. Producers enqueue work items on a bounded channel and a background worker consumes them. Application code built on `System.Threading.Channels` and `BackgroundService`.

Pattern role	Actual symbol	File and lines
Producer	`EmailService.SendEmailAsync` / `SendInvoiceEmailAsync`	`Service/EmailService.cs:8-57` (`Enqueue` at 19, 54)
Queue abstraction	`IEmailBackgroundQueue`	`Service/EmailBackgroundQueue.cs:9-13`
Bounded buffer	`EmailBackgroundQueue` (`Channel.CreateBounded<...>(100)`)	`Service/EmailBackgroundQueue.cs:15-25` (channel at 18)
Consumer	`EmailQueueHostedService.ExecuteAsync`	`Service/EmailQueueHostedService.cs:6-33` (`await foreach` at 21)
Work executor	`IEmailSender` / `MailKitEmailSender`	`Service/IEmailSender.cs:5-9`; `Service/MailKitEmailSender.cs:9-64`
Wiring	singleton queue, singleton sender, hosted service	`Program.cs:190-193`

Evidence: The producer writes a `Func<IEmailSender, CancellationToken, Task>` with `_channel.Writer.TryWrite`; the hosted service reads with `ReadAllAsync` in its own loop and executes each item with the injected sender. Producer and consumer share only the queue.

Diagram: Figure 5; source `Part1/pattern_architecture/email_producer_consumer.dot`.

Verification: Confirmed. Command was rejected for the queued delegates (no command abstraction or invoker).



Figure 5: Email background work queue.

2.3.6 Java Microservice Adapter

Pattern type and category: Adapter (GoF structural).

Purpose and origin: The legacy Java service speaks HTTP/JSON with Java `LocalDateTime` semantics. The adapter presents it to .NET code as an ordinary C# interface. Application code.

Pattern role	Actual symbol	File and lines
Target	`IJavaMicroserviceClient`	`Service/IJavaMicroserviceClient.cs: 9-14`
Adapter	`JavaMicroserviceClient`	`Service/JavaMicroserviceClient.cs:9-73`
Adaptation detail	`NaiveDateTimeConverter`, `JavaJsonOptions`	`Service/JavaMicroserviceClient.cs: 16,33`
Adaptee	Java Spring Boot backend via typed `HttpClient`	external; registered at `Program.cs:204`
Client	`JavaInvoicePdfService`, `LegacyProxyController`	see records 7 and 8

Evidence: Callers use methods such as `GetStatesAsync` and `GenerateInvoicePdfAsync`. The adapter turns them into HTTP calls (e.g. `GetAsync("/api/states")`, line 48) and converts `DateTime` values so that Jackson's `LocalDateTimeDeserializer` accepts them (comment and converter at lines 11–27).

Verification: Confirmed.

2.3.7 Legacy API Proxy

Pattern type and category: Proxy (GoF structural; remote proxy).

Purpose and origin: Exposes selected legacy Java endpoints under the .NET API so clients use one host. Application code.

Pattern role	Actual symbol	File and lines
Proxy	`LegacyProxyController` (`[Route("api/legacy")]`)	`Controllers/LegacyProxyController.cs: 14-36` (route at 12)
Forwarding calls	`_javaClient.GetStatesAsync()`, `GetHealthAsync()`	`Controllers/LegacyProxyController.cs: 26,33`
Subject interface	`IJavaMicroserviceClient`	`Service/IJavaMicroserviceClient.cs: 9-14`
RealSubject	remote Java service, reached through `JavaMicroserviceClient`	external

Evidence: Each action forwards the request unchanged and returns the remote result. The proxy concerns forwarding a whole API; the adapter (record 6) concerns translating between the two sides. They are separate collaborations.

Diagram: Figure 6; source `Part1/pattern_architecture/legacy_proxy.dot`.

Verification: Confirmed.

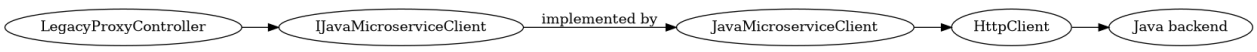


Figure 6: Legacy API proxy and Java client.

2.3.8 Legacy Java Integration

Pattern type and category: Strangler Fig (system modernisation pattern, Fowler).

Purpose and origin: The .NET API replaces the Java system incrementally; functions not yet migrated are routed to Java behind one boundary. Application code, documented as such in the source.

Pattern role	Actual symbol	File and lines
Integration boundary	`IJavaMicroserviceClient` (comment: "strangler-pattern integration")	`Service/IJavaMicroserviceClient.cs:5-14`
Routed function	`JavaInvoicePdfService` (active `IInvoicePdfService`)	`Service/JavaInvoicePdfService.cs:16-76`
Local replacement	`InvoicePdfService` (present but fully commented out)	`Service/InvoicePdfService.cs:11`
Switch point	registration with "swap back" comment	`Program.cs:194-197`
Legacy pass-through	`LegacyProxyController`	`Controllers/LegacyProxyController.cs:14-36`

Evidence: The interface comment states that the rest of the API is self-sufficient and that the Java link is opt-in. Invoice PDFs are currently produced by Java; the registration comment names the local QuestPDF implementation as the replacement. This is a system-level pattern built from the adapter and the proxy, so it is counted once and not in addition to their classes.

Verification: Confirmed.

2.3.9 API Request/Response DTO Boundary

Pattern type and category: Data Transfer Object (enterprise application pattern).

Purpose and origin: API payloads are separate from EF Core entities, so the wire format can change without changing the persistence model. Application code; the mapping uses AutoMapper.

Pattern role	Actual symbol	File and lines
Request DTO	`CreateBookingRequest`	`DTO/CreateBookingRequest.cs:7-22`
Response DTO	`BookingResponseDTO`	`DTO/BookingResponseDTO.cs:6-30`
Response DTO	`CarDTO`	`DTO/CarDTO.cs:3-26`
Assembler	`MappingProfile`; `_mapper.Map<...>` calls	`AutoMapper/MappingProfile.cs:7-34`; `Service/VehicleService.cs:30,48,92,97`
Boundary	`BookingController`, `VehicleController`, `BookingService`	records 2 and 3

Evidence: Controllers accept and return only DTO types; services map entities to DTOs (`VehicleService`) or build them explicitly (`BookingService.ToResponseDtoAsync`).

Verification: Confirmed. Data Mapper as a separate pattern is uncertain (Section 2.4).

2.4 Rejected and uncertain candidates

Candidate	Decision	Reason
Factory	rejected	Only framework creation APIs (`CreateBuilder`, `AddScoped`, `AddHttpClient`); no application-authored factory.
Singleton	rejected	`AddSingleton` is a DI lifetime; no private constructor or static instance.
Builder	rejected	`WebApplication.CreateBuilder` is framework API usage, not a custom builder.
Facade	rejected	`EmailService` simplifies the queue, but no deliberate unified interface to a subsystem is shown.
Decorator	rejected	Middleware wraps a pipeline, not objects sharing a component interface; counted as Chain of Responsibility instead.
Observer	rejected	`OnCreatingTicket` and similar are framework authentication callbacks; no application subject keeps observers.
Command	rejected	Queued `Func` delegates are command-like, but there is no command abstraction, concrete commands or invoker.
Template Method	rejected	No base class with an invariant algorithm and overridable steps.
State	rejected	`BookingStatus` and `CarStatus` are enums with conditional rules, not polymorphic state objects.
Strategy	uncertain	`IInvoicePdfService` has an active Java implementation and a local one, but the local class is commented out and there is no runtime selection.
Data Mapper	uncertain	Entity-to-DTO mapping is done by AutoMapper (framework); DTO is confirmed instead.
Unit of Work	uncertain	`DbContext` coordinates saves behind `SaveChangesAsync`, but there is no application-level `IUnitOfWork`.

State, Observer, Factory and Strategy are absent or doubtful at the baseline. Each of them is introduced by one of the Part 2 changes.

3 Part 2 Evolution and refactoring

Four changes were implemented, one per team member, each on its own branch from the baseline `dfc61ab8`. Each introduces one pattern. They are numbered here by their folder in `Part2/changes/`:

Change	Requirement	Pattern	Owner	Branch label used inside the folder
CH01	Fleet-maintenance scheduling	State	Samridhi Singh	CH03
CH02	New vehicle categories	Factory Method	Bhavana P H	CH02
CH03	Interchangeable pricing policies	Strategy	Neema Vinod	CH02
CH04	Booking lifecycle notifications	Observer	Vani Sugovind S R	CH04

Sections 3.1–3.4 describe each change applied on its own to the baseline, with the evidence recorded by its owner. Section 3.5 describes the four changes combined into the evolved application in `Part2/code/`, with the counts and metrics of every cumulative version.

Common measurement rules. Source LOC: `FlemanApi/**/*.cs` excluding `Migrations`, `bin`

and obj; a line counts if it is not blank and does not start with //. Test LOC: the same rule on FlemanApi.Tests. Tests: dotnet test on the test project with .NET SDK 8.0.425. Code coverage, cyclomatic complexity and coupling were not measured for the C# application at any version, so no value is reported.

3.1 CH01: Fleet-maintenance scheduling (State)

Change and acceptance criteria: At the baseline there is no maintenance feature. CarStatus already has UNDER_MAINTENANCE: AvailabilityRepository.CountCapacityAsync leaves such cars out of capacity, CountByCarTypeAsync counts them for the staff dashboard, and StaffService.HandoverVehicleAsync refuses any car that is not AVAILABLE. Nothing in the application ever sets a car to UNDER_MAINTENANCE except seed data.

CH01 lets staff schedule maintenance for a car (SERVICE, INSPECTION or REPAIR), start, complete or cancel it, and list what is due today or overdue. Business values chosen: a car may be scheduled for today but not for an earlier day; a car that is currently BOOKED may be scheduled, but maintenance can only start when it is AVAILABLE; "today" is the server's local calendar day; notes are limited to 500 characters.

#	Acceptance criterion	Observed result
1	Starting maintenance removes the car from CountCapacityAsync and from the handover picker, with no change to that code	Met. On EF Core InMemory with the unchanged AvailabilityRepository and StaffService, capacity goes from 2 to 1, the picker from cars {1, 2} to {2}, and the dashboard shows 1 car in maintenance. Completing restores all three.
2	Every illegal transition (e.g. complete from SCHEDULED, start from COMPLETED) gives 409 and changes nothing	Met. All 12 state × operation combinations are tested; each refusal throws ApiException 409 with schedule status, car status and timestamps unchanged, and SaveChangesAsync is never called.
3	Due and overdue lists are correct on the boundary day with a fixed clock	Met. Checked with a fixed TimeProvider at 09:30, at 00:00 and one tick before midnight.
4	Existing tests give the same result as at the baseline, with no existing test file edited	Met with a caveat: 182 of the 183 existing tests pass both before and after; the same date-dependent test fails at the baseline.
5	New tests cover each state's three operations, the service rules and the validators	Met. 50 new tests, 50 passed.

Before and after: Before: dfc61ab8dd6f9bbb2301171457dc03946a249576. After: branch ch03-maintenance-scheduling cut from the baseline; no after-commit hash was recorded and the change folder contains no diff (see Limitations). 22 files changed: 20 added, 2 modified; 1,287 lines added, 0 removed. Paths under WanderCar_DotNet/src/:

- Modified (lines only added): FlemanApi/Data/FlemanDbContext.cs, FlemanApi/Program.cs.
- Added, production: FlemanApi/Models/MaintenanceSchedule.cs; in FlemanApi/Service/Maintenance/: IMaintenanceState, MaintenanceStateBase, ScheduledState, InProgressState, CompletedState, CancelledState, MaintenanceContext; Service/IMaintenanceService.cs, Service/MaintenanceService.cs; Controllers/MaintenanceController.cs; DTOs ScheduleMaintenanceRequest, MaintenanceTransitionRequest, MaintenanceScheduleDTO; Validators/MaintenanceRequestValidators.cs.

- Added, tests: `MaintenanceStateTests`, `MaintenanceServiceTests`, `MaintenanceFleetAvailabilityTests`, `MaintenanceRequestValidatorsTests`, `TestHelpers/FixedTimeProvider.cs`.

Implementation and tests:

- Entity `MaintenanceSchedule` (table `maintenance_schedules`) with a plain `CarId`, `MaintenanceType`, `ScheduledDate`, `Status`, `StartedAt`, `CompletedAt` and `Notes`; both enums stored as strings.
- State pattern in `Service/Maintenance/`: `IMaintenanceState` (`Start`, `Complete`, `Cancel`); `MaintenanceStateBase`, which refuses every operation with `ApiException` 409 unless a subclass overrides it; four concrete states; and `MaintenanceContext`, which picks the state object from the stored `Status` with a lookup table.
- `MaintenanceService` loads the schedule and the car, passes them to the context and saves once. It has no `switch` or `if` on `Status`, and reads "now" from an injected `TimeProvider` instead of `DateTime.Now`.
- `MaintenanceController` at `api/staff/maintenance` (role STAFF) with seven endpoints, two request DTOs with FluentValidation validators; a `DbSet` and enum conversions; one commented CH01 block in `Program.cs`.

Run: `dotnet test src/FlemanApi.Tests/FlemanApi.Tests.csproj` (SDK 8.0.425). Actual result (2026-10-10): build 0 errors, 0 warnings; 233 total, 232 passed, 1 failed, 0 skipped. Baseline with the same command: 183 total, 182 passed, 1 failed. The failing test is the same date-dependent test in both runs. Logs: `Part2/changes/Change1/test_result.txt`, `Part2/changes/Change1/test_result_baseline.txt`.

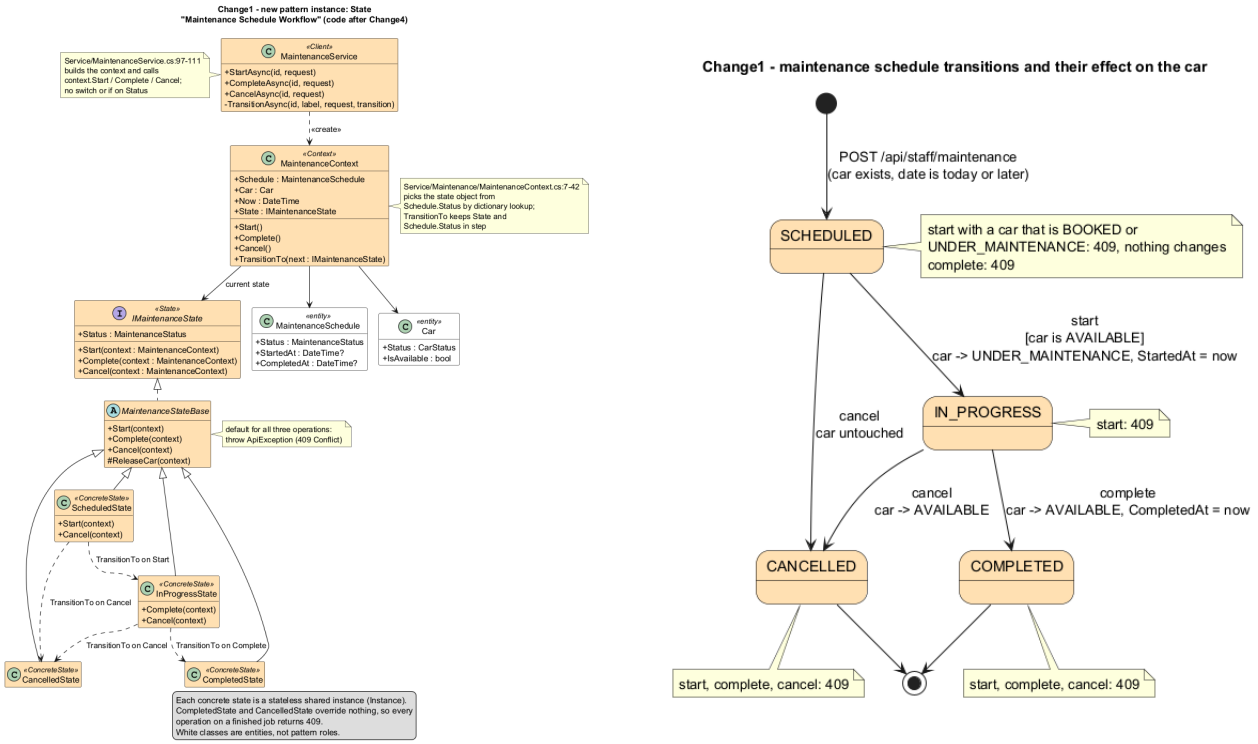


Figure 7: CH01 State roles (left) and the transitions with their effect on the car (right). Sources: `Part2/diagrams/02_change1_state_class.puml`, `03_change1_state_machine.puml`.

Pattern effects:

Instance	Effect	Reason
Maintenance Schedule Workflow (State)	newly introduced	State `IMaintenanceState`; default refusals `MaintenanceStateBase`; ConcreteStates `ScheduledState`, `InProgressState`, `CompletedState`, `CancelledState`; Context `MaintenanceContext`; Client `MaintenanceService`, which never branches on `Status`. No predecessor.
Generic Repository	extended	New entity `MaintenanceSchedule` and new client `MaintenanceService`; interface and implementation unchanged. `AvailabilityRepository` is not edited and now receives real `UNDER_MAINTENANCE` data.
ASP.NET Core DI Composition	extended	New registrations `TimeProvider.System` (singleton) and `IMaintenanceService` (scoped); new constructor-injected service and controller.
API Request/Response DTO Boundary	extended	New `ScheduleMaintenanceRequest`, `MaintenanceTransitionRequest`, `MaintenanceScheduleDTO`; no existing DTO or `MappingProfile` changed.
Request Middleware Chain	preserved	Not touched; the new 400/404/409 responses use the existing exception middleware.
Booking Service Layer; Email Background Work Queue; Java Microservice Adapter; Legacy API Proxy; Legacy Java Integration	preserved	No participant file changed.

No instance was replaced or removed. Participant file and line references for the branch are in `Part2/changes/Change1/pattern_effect.md`.

Refactoring: No existing code was refactored, and none is claimed: the two modified files gained 13 lines and lost none, and no existing test file was edited. The real smell observed is the hidden dependency on the system clock (`DateTime.Now` at `Service/StaffService.cs:91,124,149` and `Service/BookingService.cs:126`). It was left alone because injecting a clock changes the constructors of `StaffService` and `BookingService` and forces edits to existing tests. CH01 avoids the smell in its own code and registers `TimeProvider.System`, which CH03 later reuses. Behaviour-preservation evidence: no line deleted; the 183 existing tests give an identical result before and after; `AvailabilityRepository` and `StaffService` are exercised unchanged by acceptance criterion 1.

Before/after metrics:

Metric	Scope and tool	Before	After
Source LOC	common rule; find + grep + wc	3,599	3,957
Source files	same scope	107	122
Test LOC	`FlemanApi.Tests`, same rule	2,136	2,748
Tests total / passed / failed	`dotnet test`, SDK 8.0.425	183 / 182 / 1	233 / 232 / 1
Build warnings / errors	same	0 / 0	0 / 0
Lines added / removed	git numstat + `wc -l` on new files	–	1,287 / 0

Interpretation: *Benefits.* Each status's rules sit in one small class, and every illegal transition is refused in one place with the same 409. A new status such as AWAITING_PARTS needs one state class and one table entry; service, controller and validators do not change. The existing availability code began excluding cars in maintenance without a single edit. Injecting `TimeProvider` made the midnight boundary testable. *Trade-offs.* Seven types model a four-status

workflow that one `switch` could express in fewer lines. The states change the `Car` entity directly, so the workflow is coupled to it. *Limitations.* A car stops counting only when work starts, so a booking for a later date can overlap a scheduled job; schedules are not linked to bookings; there is no concurrency token. Higher LOC does not by itself show a better design; the evidence shows a workflow and a variation point added without modifying any existing line.

3.2 CH02: New vehicle categories (Factory Method)

Change and acceptance criteria: At the baseline a "category" was only a row in `car_types` with a name and three rates. The code had no category concept, no category-specific rule and no endpoint to create a car type. CH02 gives every car type a vehicle category: STANDARD (all existing car types) and the new ELECTRIC and COMMERCIAL. Rules chosen by the team: a security deposit of 0, 2× and 5× the daily rate respectively, and COMMERCIAL also requires a weekly rate. Staff can create a car type in any category; anyone can list car types by category.

#	Acceptance criterion	Observed result
1	Existing car types keep working and are reported as STANDARD with deposit 0	Met. All six existing types return STANDARD and 0 after a baseline database was upgraded.
2	`POST /api/staff/car-types` (STAFF only) creates a car type in any category and stores that category's deposit	Met. ELECTRIC at 60/day gives 120; COMMERCIAL at 80/day gives 400; STANDARD gives 0. No token gives 401; a customer token gives 403.
3	COMMERCIAL without a weekly rate gives 400; a duplicate name gives 409	Met.
4	`GET /api/car-types/category/{category}` returns only that category	Met. ELECTRIC 1, COMMERCIAL 1, STANDARD 7; an unknown category gives 400.
5	Existing tests give the same result as at the baseline, with no existing test file edited	Met with a caveat: 182 of 183 pass before and after; the same date-dependent test fails at the baseline.
6	New tests cover the three rule classes, the factory, the service and the validator	Met. 24 new tests, 24 passed.

Out of scope: pricing, booking, the return invoice and seed data. The deposit is stored and reported but not yet charged.

Before and after: Before: `dfc61ab8...`. After: `7af8d229edfaa1d5c933a0a56aa267a443bece9f` (tag `ch02`, a single commit on the baseline in the author's clone). 21 files changed: 16 added, 5 modified; 1,226 lines added, 0 removed.

- Modified (lines only added): `Models/CarType.cs`, `DTO/CarTypeDTO.cs`, `Data/FlemanDbContext.cs`, `Program.cs`, `Migrations/FlemanDbContextModelSnapshot.cs`.
- Added, production: `Models/VehicleCategory.cs`; in `Service/Categories/`: `IVehicleCategoryRules`, `StandardCategoryRules`, `ElectricCategoryRules`, `CommercialCategoryRules`, `VehicleCategoryRulesFactory`; `ICarTypeService`, `CarTypeService`; `CarTypeController`; `CreateCarTypeRequest` and its validator; migration `20261008070000_AddVehicleCategory`.
- Added, tests: `VehicleCategoryRulesTests`, `CarTypeServiceTests`, `CreateCarTypeRequestValidatorTests`.

File list: `Part2/changes/Change2/changed_files.txt`; diff: `Part2/changes/Change2/diff.patch`.

Implementation and tests: A `VehicleCategory` enum; `Category` (default STANDARD) and `SecurityDeposit` on `CarType` and `CarTypeDTO`; an `IVehicleCategoryRules` interface with one rules class per category, created by `VehicleCategoryRulesFactory.Create(category)`; `CarTypeService` and `CarTypeController` for the two endpoints; an EF Core migration adding the two columns with defaults; two DI registrations.

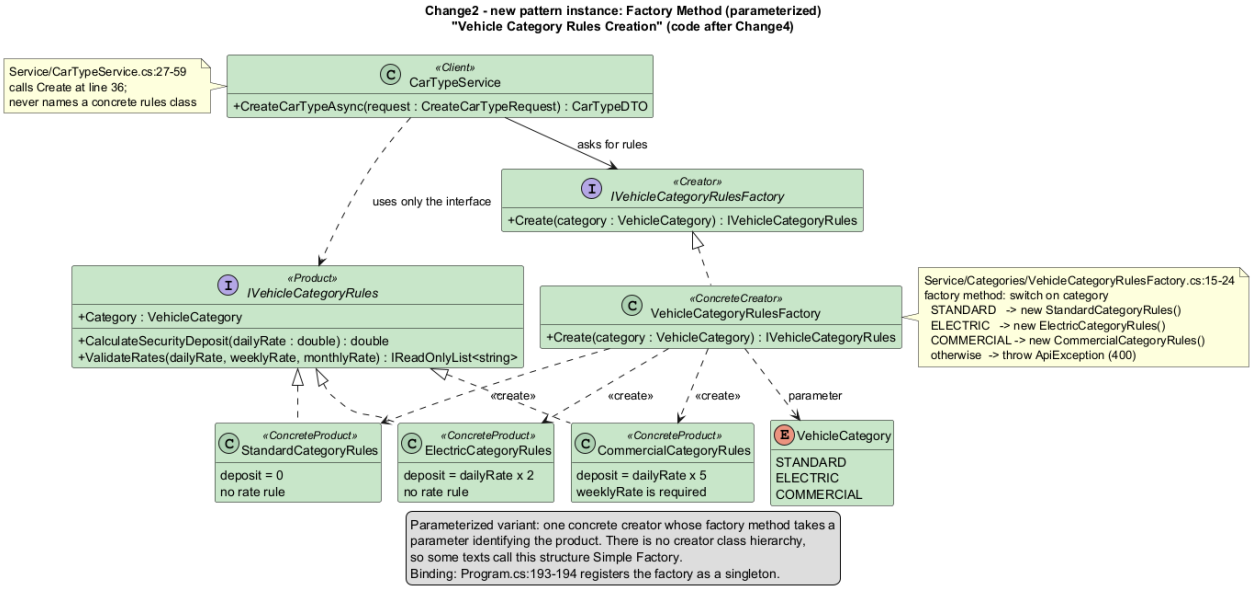


Figure 8: CH02 Factory Method roles (`Part2/diagrams/04_change2_factory_method_class.puml`).

Run: `dotnet test src/FlemanApi.Tests/FlemanApi.Tests.csproj`. Actual result: build 0 errors, 0 warnings; 207 total, 206 passed, 1 failed (the baseline test). Logs: `Part2/changes/Change2/test_result.txt`, `baseline_test.log`. Run-time check on MySQL 8.0.46: `dotnet ef migrations has-pending-model-changes` reported none; the migration was applied to a database created by the baseline build and HTTP requests exercised every criterion (`http_evidence.txt`, `db_after.txt`).

Instance	Effect	Reason

Pattern effects:

Instance	Effect	Reason
Vehicle Category Rules Creation (Factory Method, parameterized)	newly introduced	Creator `IVehicleCategoryRulesFactory` / `VehicleCategoryRulesFactory.Create`; Product `IVehicleCategoryRules`; ConcreteProducts `StandardCategoryRules`, `ElectricCategoryRules`, `CommercialCategoryRules`; Client `CarTypeService`, which never names a concrete rules class. No predecessor.
Generic Repository	extended	New client `CarTypeService`; interface and implementation unchanged.
ASP.NET Core DI Composition	extended	Two new registrations; new constructor-injected service and controller.
API Request/Response DTO Boundary	modified	`CarTypeDTO` gains `Category` and `SecurityDeposit`; new `CreateCarTypeRequest`; `MappingProfile` unchanged.
Booking Service Layer	preserved	Not touched; the new controller/service pair repeats the layering but is not part of this instance.
Middleware Chain; Email Work Queue; Java Adapter; Legacy Proxy; Legacy Java Integration	preserved	No participant file changed.

This is the parameterized variant of Factory Method (GoF, Implementation section): one creator method selects the product from a parameter. With a single concrete creator and no creator hierarchy, some texts call the structure Simple Factory. Line ranges: `Part2/changes/Change2/pattern_effect.md`.

Refactoring: None claimed; the change is additive (32 lines added to the five modified files, none removed; no existing test edited). Two smells observed and left out of scope: the hidden system clock in `StaffService.ProcessReturnAsync` (fitting technique: inject a clock) and duplicated `DaysBetween` in `BookingService` and `StaffService` (fitting technique: Extract Method). One design decision avoided a refactoring: the staff-only endpoint went into a new `CarTypeController`, because `VehicleController` carries a class-level `[AllowAnonymous]` that would override `[Authorize]`. Behaviour preservation: 0 deleted lines; identical results for the 183 existing tests; existing endpoints return the same data on an upgraded database.

Before/after metrics:

Metric	Scope and tool	Before	After
Source LOC	common rule	3,599	3,794
Source files	same scope	107	118
Test LOC	same rule	2,136	2,383
Tests total / passed / failed	`dotnet test`, SDK 8.0.425	183 / 182 / 1	207 / 206 / 1
Build warnings / errors	same	0 / 0	0 / 0
Lines added / removed	`git diff -numstat baseline ch02`	–	1,226 / 0

Of the 1,226 added lines, 645 are EF Core migration files and 309 are tests.

Interpretation: *Benefits.* Rules that differ by category sit in one small class each, and the category-to-class mapping is in one `switch`. A new category needs one enum member, one rules class and one switch arm; `CarTypeService`, the controller and the validator do not change. The rules classes have no dependencies, so they are tested without mocks. *Trade-offs.* 195 more source lines and 11 more files for rules a single `if` could express today; the switch must still be edited

per category, so clients are closed to change but the factory is not. *Limitations.* The deposit is computed once at creation and stored; it is not yet used in booking or invoicing; there is no endpoint to change an existing car type's category.

3.3 CH03: Interchangeable pricing policies (Strategy)

Change and acceptance criteria: At the baseline, prices are calculated by the static `RentalRateCalculator.CalculateRentalAmount(CarType, int)` in `Util/RentalRateCalculator.cs`, which packs whole months, then whole weeks, then remaining days. `BookingService` and `StaffService` call it directly. CH03 introduces interchangeable pricing policies with the Strategy pattern. Acceptance criteria:

1. A common contract `IPricingPolicy` exists.
2. `StandardPricingPolicy` keeps the existing month/week/day calculation.
3. `WeekendPricingPolicy` applies the configured weekend surcharge when a rental includes a Saturday or Sunday.
4. `SeasonalPricingPolicy` applies the configured surcharge when the pickup month is a peak month.
5. `PricingPolicySelector` selects the policy with precedence seasonal, then weekend, then standard.
6. Booking creation/modification and the staff return invoice use the same selector.
7. Pricing values are bound from configuration (`PricingSettings`) and everything is registered in DI.
8. Standard behaviour is preserved and pricing tests are added; no schema change.

All eight were met; evidence below.

Before and after: Before: `dfc61ab8...`. After: implemented on branch `ch02-pricing-policies` in the author's working tree; no after-commit hash was recorded (`Part2/changes/Change3/after_commit.txt`). The diff is `Part2/changes/Change3/diff.patch`: 10 new and 7 modified files. Production files: new `IPricingPolicy`, `IPricingPolicySelector`, `PricingPolicySelector`, `PricingSettings`, `RentalDurationCalculator`, `StandardPricingPolicy`, `WeekendPricingPolicy`, `SeasonalPricingPolicy` (all in `FlemanApi/Util/`); modified `Service/BookingService.cs`, `Service/StaffService.cs`, `Program.cs`; `RentalRateCalculator` retained. Tests: new `StandardPricingPolicyTests`, `PricingPolicyTests`; constructor wiring updated in `BookingServiceTests` and `StaffServiceTests`.

Implementation and tests: The selector chooses seasonal pricing when the pickup month is configured as a peak month (default December, January, April; +30%); otherwise weekend pricing (+20%) when the interval contains a Saturday or Sunday; otherwise standard. `PricingSettings` is bound with `IOptions`; `BookingService` and `StaffService` receive `IPricingPolicySelector` by constructor injection. `StaffService` also receives an injected `TimeProvider`, and `StaffServiceTests` now use a fixed clock. Run: `dotnet test .\WanderCar_DotNet\src\FlemanApi.Tests`. Result recorded by the author: 205 total, 205 passed, 0 failed (transcribed terminal output, `Part2/changes/Change3/test_result.txt`); filtered runs: 22 pricing-policy tests and 17 standard-pricing tests passed. The previously failing date-dependent test passes because of the injected clock. Independent re-run on the integrated code: 279 of 279 after this step (Section 3.5).

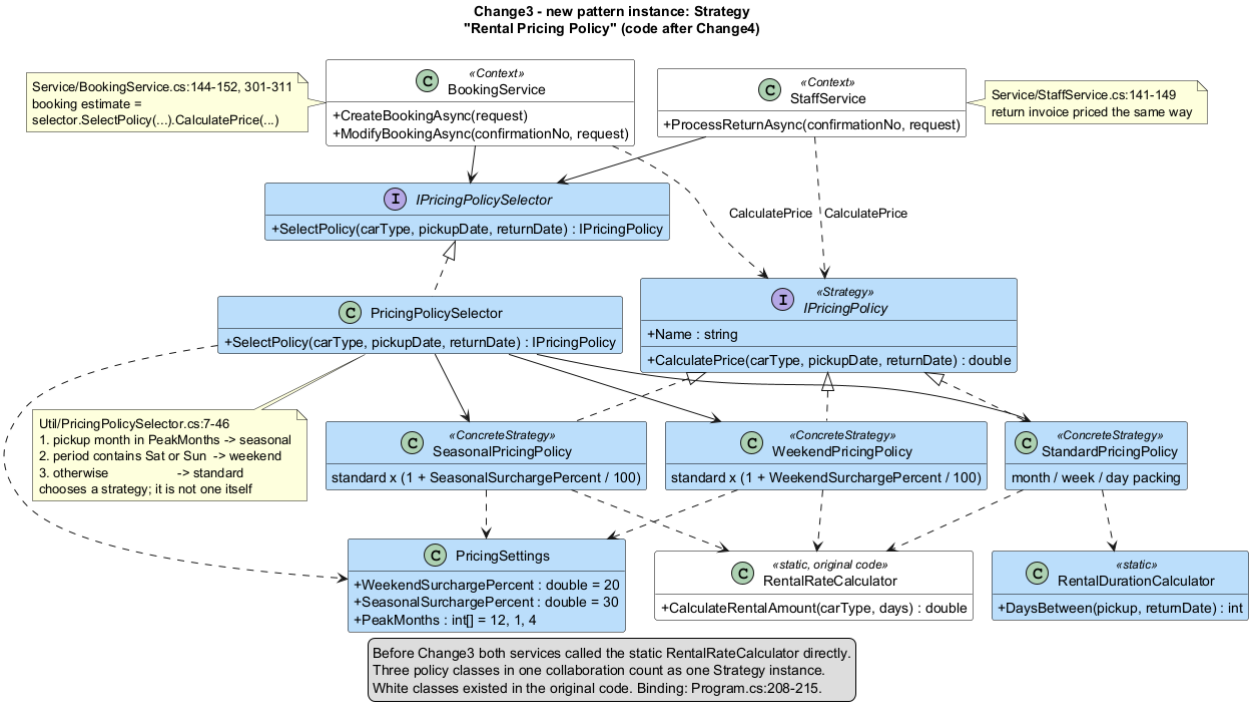


Figure 9: CH03 Strategy roles, selector and the two calling services (`Part2/diagrams/05_change3_strategy_class.puml`).

Pattern effects:

Instance	Effect	Reason
Rental Pricing Policy (Strategy)	newly introduced	Strategy `IPricingPolicy`; ConcreteStrategies `StandardPricingPolicy`, `WeekendPricingPolicy`, `SeasonalPricingPolicy`; Contexts `BookingService` and `StaffService`; `PricingPolicySelector` chooses the strategy and is not counted as a separate instance. It makes concrete the "strategy-capable" idea that was only uncertain at the baseline.
Booking Service Layer	modified	`BookingService` gains `IPricingPolicySelector` and prices create/modify through it; `IBookingService` unchanged.
ASP.NET Core DI Composition	extended	`PricingSettings` options binding, three policies, the selector and a single `TimeProvider` registration.
All other baseline instances	preserved	No participant changed. `RentalRateCalculator` is kept as the standard calculation.

Refactoring: *Smell:* services were coupled to a static calculator, so a new pricing rule meant editing both services, and `StaffService` depended on the system clock. *Technique:* Extract Interface (`IPricingPolicy`), encapsulate each algorithm in a policy class (Replace Conditional with Polymorphism), centralise the choice in `PricingPolicySelector`, and inject configuration and the clock. *Behaviour preservation:* `StandardPricingPolicy` delegates to the unchanged `RentalRateCalculator`; `RentalRateCalculatorTests` remain; standard-equivalence tests pass. `RentalDurationCalculator.DaysBetween` was added, but the private `DaysBetween` methods in the two services remain, so this duplication is not claimed as removed.

Before/after metrics: The author measured a pricing-focused scope (whole files, blank lines included), so these figures must not be compared with the common-rule LOC of the other changes; the common-rule figures for the cumulative code are in Section 3.5.

Measure	Before	After	Scope
Production LOC, pricing-related files	1,039	1,313	includes full `BookingService`, `StaffService`, `Program.cs`
Pricing utility LOC	36	215	`RentalRateCalculator` vs. calculator plus seven new files
Pricing test LOC	99	337	calculator tests vs. plus two new test files
Tests total / passed / failed	183 / 182 / 1	205 / 205 / 0	full suite
Pricing-policy tests	–	22 passed	filtered run

Recorded suite durations (9 s before, 4 s after) are single runs and do not show a performance change.

Interpretation: *Benefits.* Pricing variants are explicit objects chosen in one place, and booking and return invoice can no longer disagree on the rule. The injected clock removed the only failing test. *Trade-offs.* More types and registrations. *Limitations.* The weekend surcharge applies to the whole rental when any weekend day occurs; seasonal selection checks the pickup month only; no commit hash was recorded.

3.4 CH04: Booking lifecycle notifications (Observer)

Change and acceptance criteria: At the baseline, `BookingService.CreateBookingAsync` built the confirmation e-mail itself and called `IEmailService.SendEmailAsync` directly inside a `try/catch` that logged and swallowed errors (`Service/BookingService.cs:167-196`). `ModifyBookingAsync` and `CancelBookingAsync` notified nobody, and no booking event was audited. CH04 notifies the customer when a booking is created, modified or cancelled and writes one audit entry per event, using the Observer pattern so that new reactions need no edit to `BookingService`. Business values: the CREATED e-mail keeps the baseline subject and body; MODIFIED and CANCELLED use "Booking Updated - Fleeman" and "Booking Cancelled - Fleeman"; events are published only after a successful save; a failing observer is logged and skipped and never changes the HTTP result; the audit entry goes to `ILogger`.

#	Acceptance criterion	Observed result
1	Creating a booking still queues one confirmation e-mail with the baseline recipient, subject and body	Met. Tests check the queued work item and compare subject and body with the baseline text.
2	Modifying and cancelling each queue one e-mail and write one audit entry	Met. A rejected modification publishes nothing.
3	An observer failure does not change the HTTP result and remaining observers still run	Met.
4	Existing tests keep their baseline result; the only edit to `BookingServiceTests` is the constructor argument	Met with a caveat: 182/183 before, 200/201 after; the same date-dependent test fails in both.
5	New tests cover subscription, unsubscription, ordering, failure isolation and both observers	Met. 18 new tests, 18 passed.

Before and after: Before: `dfc61ab8...`. After: `9be38a6ff4cd5b10b813aedb42c3de8a1e82730d` (tag `ch04`, a single commit on the baseline in a local clone). 13 files changed: 10 added, 3

modified; 709 lines added, 34 removed. Modified: `Service/BookingService.cs`, `Program.cs`, `FlemanApi.Tests/Services/BookingServiceTests.cs` (constructor argument only). Added in `Service/Notifications/`: `BookingEvent`, `IBookingEventObserver`, `IBookingEventPublisher`, `BookingEventPublisher`, `EmailBookingObserver`, `AuditLogBookingObserver`; tests `BookingEventPublisherTests`, `BookingObserverTests`, `BookingServiceNotificationTests`, `TestHelpers/ListLogger.cs`. Diff: `Part2/changes/Change4/diff.patch`.

Implementation and tests: `BookingEvent` is an immutable record (kind, booking and customer data), a snapshot so observers never receive the tracked EF entity. `BookingEventPublisher` receives all `IBookingEventObserver` registrations from DI, subscribes them in order, and notifies them sequentially over a copy of its list, isolating each in a `try/catch`. `EmailBookingObserver` sends through the unchanged `IEmailService`; `AuditLogBookingObserver` writes a structured `ILogger` entry. `BookingService` receives `IBookingEventPublisher` instead of `IEmailService` and publishes after each successful save. No migration. Run: `dotnet test src/FlemanApi.Tests/FlemanApi.Tests.csproj`. Actual result (2026-10-10): build 0 errors, 0 warnings; 201 total, 200 passed, 1 failed (the baseline test); all 21 existing `BookingServiceTests` pass. Logs: `Part2/changes/Change4/test_result.txt`, `test_result_baseline.txt`.

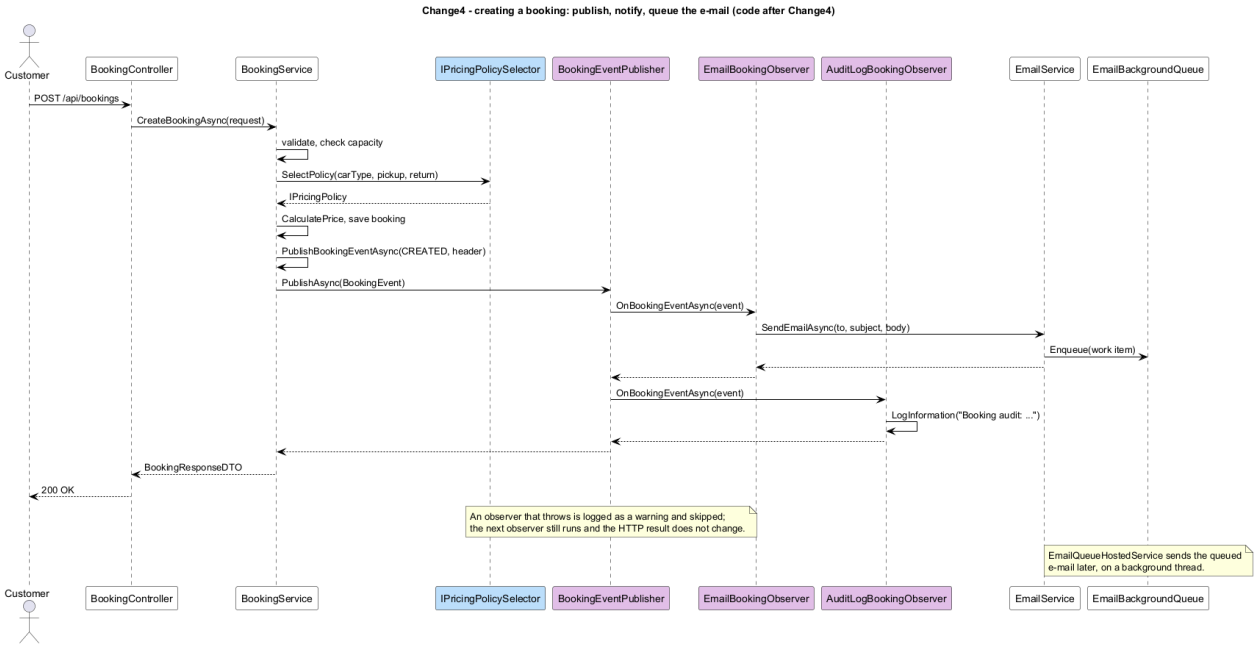


Figure 10: CH04: creating a booking publishes an event that both observers handle (`Part2/diagrams/07_change4_observer_sequence.puml`; class view in `06_change4_observer_class.puml`).

Instance	Effect	Reason

Pattern effects:

Instance	Effect	Reason
Booking Lifecycle Notification (Observer)	newly introduced	Subject `IBookingEventPublisher`; ConcreteSubject `BookingEventPublisher`; Observer `IBookingEventObserver`; ConcreteObservers `EmailBookingObserver`, `AuditLogBookingObserver`; push-model state `BookingEvent`. Replaces the framework-hook "Observer" rejected in Part 1 with an application-authored one.
Booking Service Layer	modified	Loses inline e-mail logic and `IEmailService`; gains `IBookingEventPublisher` and `PublishBookingEventAsync`.
ASP.NET Core DI Composition	extended	Three registrations; observers resolved as `IEnumerable< IBookingEventObserver>`, registration order = notification order.
Email Background Work Queue	preserved	Unchanged; `EmailBookingObserver` becomes a new producer-side caller.
Generic Repository	preserved	Unchanged; `BookingService` makes extra reads to build the event.
DTO Boundary; Middleware Chain; Java Adapter; Legacy Proxy; Legacy Java Integration	preserved	No participant changed. `BookingEvent` is internal, not an API DTO.

Candidates rejected for this code: Mediator (observers do not coordinate through the publisher) and Chain of Responsibility (every observer runs; none stops the others).

Refactoring: *Smell:* Long Method with mixed responsibilities. `CreateBookingAsync` (123 lines) ended with 30 lines that reloaded the customer, formatted the e-mail and sent it in its own `try/ catch`; adding modify/cancel notifications would have copied that block twice. *Technique:* Extract Class + Move Method into `EmailBookingObserver` (literal text unchanged), then replace the direct call with `PublishBookingEventAsync`. Diff hunk `@@ -164,36 +165,7 @@` in `Part2/ changes/Change4/diff.patch`. *Behaviour preservation:* all 21 existing `BookingServiceTests` pass; `EmailObserver_Created_SendsExactlyTheBaselineSubjectAndBody` and `CreateBookingAsync_QueuesOneConfirmationEmail_WithBaselineRecipientSubjectAndBody` compare against the baseline text.

Before/after metrics:

Metric	Scope and tool	Before	After
Source LOC	common rule	3,599	3,739
Source files	same scope	107	113
Test LOC	same rule	2,136	2,508
Tests total / passed / failed	`dotnet test`, SDK 8.0.425	183 / 182 / 1	201 / 200 / 1
Build warnings / errors	`dotnet build -no-incremental`	0 / 0	0 / 0
Lines added / removed	`git diff -shortstat`	–	709 / 34

Of the 34 removed lines, 33 are in `BookingService.cs` (the moved e-mail block and the `IEmailService` field, parameter and assignment) and one is the replaced constructor argument in the test.

Interpretation: *Benefits.* `BookingService` publishes events without knowing which reactions run; a new reaction is one observer class and one registration. Creation keeps the baseline e-mail;

modify and cancel gain notifications; observer failures are isolated. *Trade-offs.* Six production files and 140 source lines; observers run synchronously in the request and event construction needs extra reads. *Limitations.* Audit entries go to the log, not a durable table; handover and return are not covered; e-mail code in `CustomerService` and `StaffService` is unchanged.

3.5 Integration and updated pattern counts

The four changes were applied to the baseline in folder order to produce the evolved application in `Part2/code/WanderCar_DotNet`. One patch per step is in `Part2/changes/integration/`. Three small edits were needed to make them work together: `TimeProvider` is registered once (CH01 and CH03 both registered it); the CH01 test that builds a `StaffService` passes the two new constructor arguments from CH03; the CH04 test that builds a `BookingService` passes a selector returning `StandardPricingPolicy`. Step 1 (CH01) was rebuilt from the CH01 documents and its 50 test names, because the change folder has no diff (see Limitations).

Snapshot	Types	Inst.	Concrete participant classes added	Rejected / uncertain observations
Original code `dfc61ab8`	9	9	15 distinct (Part 1)	9 rejected, 3 uncertain (Part 1)
After CH01	10	10	`ScheduledState`, `InProgressState`, `CompletedState`, `CancelledState` (+ `MaintenanceContext`)	Rejected: Singleton (the states' shared `Instance` fields are a State implementation detail); Template Method (`MaintenanceStateBase` supplies default refusals, not an algorithm skeleton).
After CH02	11	11	`VehicleCategoryRulesFactory`, `StandardCategoryRules`, `ElectricCategoryRules`, `CommercialCategoryRules`	Uncertain: Factory Method vs. Simple Factory label. Rejected: `CarTypeController`/`CarTypeService` as a second Service Layer instance.
After CH03	12	12	`StandardPricingPolicy`, `WeekendPricingPolicy`, `SeasonalPricingPolicy`	The selector is not a separate Strategy instance. Baseline "Strategy (uncertain)" on `IInvoicePdfService` is unchanged.
After CH04	13	13	`BookingEventPublisher`, `EmailBookingObserver`, `AuditLogBookingObserver`	Rejected: Mediator, Chain of Responsibility. Uncertain: Command on the e-mail queue (unchanged).

All nine baseline instances keep their names because their collaborations continue. Each new instance (Maintenance Schedule Workflow, Vehicle Category Rules Creation, Rental Pricing Policy, Booking Lifecycle Notification) has no predecessor and replaces nothing. No instance was removed. Per-version rows with participants, line ranges in the evolved code and status are in `Part2/pattern_evaluation.csv` (55 rows); counts in `Part2/Pattern_count.csv`.

Version (cumulative)	Source LOC	Files	Test LOC	Tests	Passed	Failed
Original code	3,599	107	2,136	183	182	1
After CH01	3,956	122	2,758	233	232	1
After CH02	4,151	133	3,005	257	256	1
After CH03	4,360	141	3,225	279	279	0
After CH04 (`Part2/code`)	4,500	147	3,603	297	297	0

All five rows were measured on 2026-10-11 with the common rule and SDK 8.0.425; the final log is `Part2/code/test_result.txt` (build 0 warnings, 0 errors). Original to final: 67 files changed (58 added, 9 modified), 4,545 lines added and 52 removed, including tests and migra-

tions. The failure disappears at CH03 because `StaffService` then takes its time from an injected `TimeProvider`; no change broke or accidentally repaired behaviour.

Instance	Original	CH01	CH02	CH03	CH04
Generic Repository	baseline	extended	extended	preserved	preserved
ASP.NET Core DI Composition	baseline	extended	extended	extended	extended
Booking Service Layer	baseline	preserved	preserved	modified	modified
API Request/Response DTO Boundary	baseline	extended	modified	preserved	preserved
Middleware Chain, Email Queue, Adapter, Proxy, Strangler Fig	baseline	preserved	preserved	preserved	preserved
Maintenance Schedule Workflow (State)	–	new	preserved	preserved	preserved
Vehicle Category Rules Creation (Factory Method)	–	–	new	preserved	preserved
Rental Pricing Policy (Strategy)	–	–	–	new	preserved
Booking Lifecycle Notification (Observer)	–	–	–	–	new

Table 10: Status of every instance after each cumulative change (from `Part2/pattern_evaluation.csv`).

Interpretation: The patterns paid off where change happened: each new requirement was delivered mostly as new classes plus registrations, and the existing collaborations survived intact (DI was extended by every change; the Booking Service Layer was modified twice without changing `IBookingService`). Two real refactorings removed real smells (CH03: static pricing coupling and the hidden clock; CH04: a long, mixed-responsibility method). The cost is more types to navigate: source LOC grew 25% and test LOC 69%. More lines do not by themselves prove a better design; the evidence supports the narrower claim that each variation point was added without breaking existing behaviour, as shown by the unchanged results of the original tests.

4 Part 3 Cross-language comparison

Selected instances:

Pattern type	Source instance	Selection reason
Factory Method (parameterized)	Vehicle Category Rules Creation (CH02, `Service/Categories/`, commit `7af8d229`)	Verified in CH02; self-contained (no framework, DB or DI dependency); exercises interfaces, enumerations, optional values and unknown-value handling.
Observer	Booking Lifecycle Notification (CH04, `Service/Notifications/`, commit `9be38a6f`)	The application's only Observer; few participants; testable without a database; ordering and failure isolation are well defined.
Dependency Injection / IoC	ASP.NET Core DI Composition (baseline, as extended by CH01; `Program.cs:179-214`)	Composition root of the whole application; in C# the container is framework-provided, so the ports must rebuild it, exposing run-time type information.
Strategy	Rental Pricing Policy (CH03)	Smoke-level port only (see below).
Repository	Generic Repository (baseline)	Smoke-level port only (see below).

The remaining baseline types (Service Layer, Middleware Chain, Producer–Consumer, Adapter, Proxy, Strangler Fig, DTO) and State were not ported; this is a limitation of Part 3.

Common measurement rules: Each pattern has one `common_spec.md` that every port imple-

ments exactly. **Source LOC**: the snippet file only (test runner, README and scripts excluded), non-blank lines that are not comment-only; C++ preprocessor lines count. **Maximum function cyclomatic complexity**: 1 plus one per `if`, loop, `case`, `catch`/`except`, `&&`/`and`, `||`/`or`; counted by hand for Factory Method and Observer and with `lizard` 1.24.1 for DI. **Median process time**: wall-clock time of the whole test-runner process after compiling, so it *includes runtime start-up*; Factory Method and Observer: one warm-up and five measured runs; DI: eleven runs, no warm-up. Raw values are in each folder's `timings.txt` or `measure_output.txt`. Different patterns were measured by different members on different machines, so times are comparable across languages within a pattern only.

4.1 Factory Method: Vehicle Category Rules Creation

Common specification: Participants: enumeration `VehicleCategory` (STANDARD, ELECTRIC, COMMERCIAL); a rules abstraction with `category()`, `calculateSecurityDeposit(dailyRate)` and `validateRates(dailyRate, weeklyRate, monthlyRate)`; three concrete rules classes; a factory with `create(category)`. Rules: deposit 0, 2× and 5× daily rate; COMMERCIAL without weekly rate returns exactly one error "weeklyRate is required for COMMERCIAL car types". `create` is the only place that names concrete classes and rejects an unknown category with an exception. Seven cases, identical in every language: (1) STANDARD deposit 0 and no errors; (2) ELECTRIC 60 gives 120; (3) COMMERCIAL 80 gives 400; (4) COMMERCIAL without weekly rate gives the error; (5) with weekly rate gives none; (6) every `create(c).category()` equals `c`; (7) unknown category rejected. No fixtures or test framework; each runner prints PASS/FAIL per case and exits non-zero on failure. Specification: `Part3/Factory_Method/common_spec.md`.

Implementations and actual results: All under `Part3/Factory_Method/`, standard library only, run on 2026-10-10.

Language	Snippet (test runner)	Version	Result
Java	`Java/VehicleCategoryRulesFactory.java` (`TestRunner.java`)	javac / OpenJDK 17.0.16	7/7
Python	`Python/vehicle_category_rules.py` (`test_runner.py`)	CPython 3.14.3	7/7
JavaScript	`Javascript/vehicleCategoryRules.js` (`testRunner.js`)	Node.js v24.14.1	7/7
C++	`Cpp/vehicle_category_rules.hpp` (`test_runner.cpp`)	g++ 15.2.0 (MSYS2), C++17	7/7

Commands, run inside each language folder:

```
Java        javac VehicleCategoryRulesFactory.java TestRunner.java
            java -cp . TestRunner
Python      python test_runner.py
JavaScript  node testRunner.js
C++         g++ -std=c++17 -O2 -Wall -Wextra -static \
                -o test_runner.exe test_runner.cpp
            test_runner.exe
```

Each log (`test_result.txt` in the language folder) is the unedited output of a real run; exit code 0 in all four; Java and C++ compiled without warnings.

Static and dynamic comparison:

Language	LOC	Max CC	Runtime / compiler	Tests	Median ms
Java	65	4	OpenJDK 17.0.16	7/7	109.8
Python	51	2	CPython 3.14.3	7/7	56.6
JavaScript	77	4	Node.js v24.14.1	7/7	70.1
C++	71	4	g++ 15.2.0, `-O2`	7/7	14.6

Comparison: The participant structure is identical in all four and matches the C# source: one enumeration, one product abstraction, three concrete products, one concrete creator with a parameterized `create`. *Abstraction:* Java interface (missing method = compile error); C++ abstract class with pure virtual functions and virtual destructor (also compile-time); Python `abc.ABC` (checked at construction); JavaScript has no interface, so a base class whose methods throw "not implemented" documents the duck-typed contract. *Selecting the product:* Java's `switch` expression over an enum must be exhaustive; C++ and JavaScript use a `switch` statement; Python stores classes in a dictionary, which is why its CC is 2. *Absent value:* boxed `Double`/`null`, `None`, `null`/`undefined`, `std::optional<double>`. *Ownership:* only C++ states it, returning `std::unique_ptr`. *Unknown category:* a Java enum can only be `null`; a C++ `enum class` can hold `999`; Python and JavaScript accept any value (here the string `'HOVERCRAFT'`). Python is shortest because of the dictionary and no type declarations; JavaScript is longest because of the documenting base class and exports. Timings are dominated by JVM, interpreter or Node start-up.

4.2 Observer: Booking Lifecycle Notification

Common specification: Participants: `BookingEventKind` (CREATED, MODIFIED, CANCELLED); `BookingEvent` (kind, confirmation number, customer name, optional e-mail, car type, estimated amount); `BookingEventObserver` with `onBookingEvent`; `BookingEventSubject` with `subscribe`, `unsubscribe`, `publish`; `BookingEventPublisher`; `EmailBookingObserver` (appends to an in-memory outbox; nothing if no e-mail) and `AuditLogBookingObserver`. Rules: initial observers subscribed in order; duplicate subscription of the same object ignored (identity); notification once each, in order, over a copy; a throwing observer adds exactly one warning and the rest still run; `publish` never throws. Notification is synchronous and side effects are in-memory lists, which keeps the ports comparable while preserving the structure. Eight cases cover the three kinds, order, duplicates, unsubscription, failure isolation and a missing e-mail. Specification: `Part3/Observer_Pattern/common_spec.md`.

Implementations and actual results: All under `Part3/Observer_Pattern/`, no third-party libraries, run on 2026-10-10.

Language	Tool / version	Command	Result
Java	javac / HotSpot 26.0.1	`javac BookingEventPublisher.java TestRunner.java && java -cp . TestRunner`	8/8, exit 0
Python	CPython 3.14.6	`python test_runner.py`	8/8, exit 0
JavaScript	Node.js v24.18.0	`node testRunner.js`	8/8, exit 0
C++	clang 21.1.0 via zig 0.16.0	`python -m ziglang c++ -std=c++17 -O2 -Wall -Wextra -o test_runner.exe test_runner.cpp`	8/8, exit 0

Logs: `Part3/Observer_Pattern/<Language>/test_result.txt`; combined measurement `Part3/Observer_Pattern/measure_output.txt`.

Static and dynamic comparison:

Language	LOC	Max CC	Runtime / compiler	Tests	Median ms
Java	71	9	HotSpot 26.0.1	8/8	238.7
Python	69	3	CPython 3.14.6	8/8	65.5
JavaScript	63	3	Node.js v24.18.0	8/8	71.0
C++	102	6	clang 21.1.0, `-O2`	8/8	16.9

Comparison: All four use a publisher with an observer collection, an observer abstraction and two concrete observers. *Abstraction:* Java interfaces checked at compile time (a single-method observer interface lets lambdas act as observers); Python ABCs, though `publish` works with any object that has `on_booking_event`; JavaScript base classes and plain object literals; C++ abstract classes. *Identity:* `==`, `is`, `includes`/`!==`, `shared_ptr` comparison. *Ownership:* garbage collection vs. explicit `shared_ptr` (the subject keeps observers alive; `weak_ptr` would be the non-owning alternative). *Failure isolation:* Java catches `RuntimeException`, Python `Exception`, JavaScript any thrown value, C++ `const std::exception&` (other types would escape). Node's `EventEmitter` was not used because a throwing listener stops the remaining ones, which breaks case 7. C++ is longest because `enum class` needs a hand-written `to_string`; Java's maximum CC is in `EmailBookingObserver.onBookingEvent` (an `if`, an `||` and two exhaustive switches), whereas Python looks subject and verb up in a dictionary.

4.3 Dependency Injection: Service Registration and Constructor Injection

Common specification: Participants: `Lifetime` (SINGLETON, SCOPED, TRANSIENT); a `ServiceCollection` (`addSingleton`, `addScoped`, `addTransient`, `addSingletonInstance`, `addOpenGeneric`); a `ServiceProvider` acting as root and scope (`createScope`, `getRequiredService(token)`); services identified by string tokens. Rules as in .NET: singletons built once and shared; scoped once per scope; transient on every request; an open-generic `IGenericRepository` resolves `IGenericRepository<Car>`; the last registration wins. Errors with the .NET messages: "No service for type 'X' has been registered." and "Cannot resolve scoped service 'X' from root provider." Ten cases (D01–D09 with D04b) use small stand-ins for `FlemanDbContext`, `GenericRepository`, a fixed clock and `MaintenanceService`; no file, database or network access.

Implementations and actual results: All under `Part3/Dependency_Injection_Pattern/`, built and run on 2026-10-10.

Language	Tool / version	Test command	Result
Java	javac/java 24.0.2; JUnit Platform 1.11.4	`java -jar junit-platform-console-standalone-1.11.4.jar execute -select-class di.DependencyInjectionTest`	10/10
Python	CPython 3.13.7; unittest	`python -m unittest test_dependency_injection`	10/10
JavaScript	Node.js v22.18.0; node:test	`node -test dependency_injection.test.mjs`	10/10
C++	clang 21.1.0 via zig 0.16.0	`python -m ziglang c++ -std=c++17 -O2 -Wall -Wextra ...`	10/10, 0 warnings

Java failed case D04b on its first run: `addSingleton("X", Foo.class)` matched an `addSingleton(String, Object)` overload (Java prefers a non-varargs overload), so the `Class` object itself was registered. The instance overload was renamed `addSingletonInstance`; the result above is after that fix. To show that the tests can fail, two deliberate bugs were injected into a temporary copy in each language; all eight were caught. These runs were made in the author's

folder layout; the test logs were not uploaded, and in this repository the JavaScript test imports `../src/`, which must be corrected before re-running.

Static and dynamic comparison:

Language	LOC	Max CC	Runtime / compiler	Tests	Median ms
Java	110	7	Java 24.0.2	10/10	852.8
Python	81	8	CPython 3.13.7	10/10	132.0
JavaScript	57	8	Node.js v22.18.0	10/10	153.8
C++	122	7	clang 21.1.0, `-O2`	10/10	9.9

Python LOC counts docstring lines (`lizard` reports 67 without them). Java's figure is JVM plus JUnit launcher start-up (fastest run 821 ms).

Comparison: The roles match the .NET container: a registration object (the `Program.cs` role), a provider that is both root (owning singletons) and scope (owning scoped instances), and constructor-injected clients. No implementation inheritance is used; the provider is composed of registration records and caches. *Building an object from its dependencies:* Java reflection (`Constructor.newInstance`), Python `impl(*deps)`, JavaScript `new Impl(...deps)`, C++ a variadic template `make_shared<Impl>(get<Deps>(token)...)` so a wrong constructor is a compile error. Java, Python and JavaScript list dependency tokens in the registration, because their constructors carry no usable run-time type information. *Keys:* string tokens everywhere; in Java type erasure makes `IGenericRepository<Car>` and `<Hub>` the same `Class`; C++ also stores `std::type_index` and checks it before casting. *Ownership:* garbage collection vs. `shared_ptr`, with the root required to outlive its scopes. CC is 7–8 everywhere because the resolve method holds the same lifetime and scope checks. None of the ports is thread-safe, unlike the .NET container.

4.4 Strategy and Repository (smoke-level ports)

`Part3/strategy_pattern/` and `Part3/repository_pattern/` each hold one small runnable program per language (`Main.java`, `main.py`, `main.js`, `main.cpp`) without a separate test runner. Checked on 2026-10-11 with JDK 17.0.16, CPython 3.14.3, Node.js 24.14.1 and g++ 15.2.0: all four Strategy programs end with `Weekend:  360.00` (policy interface, standard/weekend/seasonal policies and a selector with seasonal precedence), and all four Repository programs print `Repository CRUD: PASS` (map-backed generic CRUD). Their LOC, CC and timing were not measured to the common rules, so they are not included in the comparison tables.

Technology discussion: Gamma et al. describe the parameterized factory method as one in which "the factory method takes a parameter that identifies the kind of object to create"; Fowler defines Dependency Injection as a form of Inversion of Control in which a separate assembler supplies an object's dependencies, with constructor injection as a main form; the Observer pattern defines a one-to-many dependency in which dependents are notified of changes, here in the push model. Across the three measured patterns, the same roles appear in all four languages, while the type system decides how much of the pattern is enforced and when a mistake is caught. In Java and C++ the compiler catches a missing product, a missing interface method or (in C++) a wrong DI constructor; in Python and JavaScript the same structures are conventions enforced by tests. Python even reduces Factory Method to a dictionary of classes, because classes are first-class values. Related platform mechanisms (.NET `IObservable<T>`, Node `EventEmitter`, the ASP.NET Core container) were deliberately not used, because the shared specifications require behaviour (ordering, failure isolation, explicit lifetimes) that must be identical in every port.

5 LLM records and human verification

LLMs were used as assistants; every claim was checked against source code or a re-run. Records are in `Part1/llm.csv`, `Part2/llm.csv` and `Part3/llm/llm.csv`.

Run and model:

Part	Displayed model, date	Task	Evaluation and correction
Part 1	GPT-5.6 Luna, 2026-10-08	Pattern discovery on the extracted source context	**Partially correct.** 9 confirmed, 3 uncertain, 9 rejected candidates proposed; each confirmed one was checked manually against the source. Its output contained no line ranges; those in this report were taken from the baseline files.
Part 2	Claude Opus 5.5 (Claude Code), 2026-10-10	CH04 Observer implementation and evidence	**Correct, with corrections.** Build and tests re-run (201/200/1); every role in `pattern_effect.md` maps to a real class; LOC recomputed. Corrections: the `IEmailService` removal was decided by the model, not agreed first; the commit hash is local only.
Part 2	Claude Opus 5.5, 2026-10-11	Integration of CH01–CH04 into `Part2/code`	**Correct for what was executed.** Patches apply cleanly; build clean; 297/297. Not verified: the evolved application was not run against MySQL.
Part 2	Claude Opus 5.5, 2026-10-11	Rebuilding CH01 code from its documents (no diff existed)	**Cannot confirm** as the team's original code: all names, routes and rules match and the 50 test names pass, LOC 3,956 vs. 3,957 reported; method bodies and line numbers may differ.
Part 2	Claude Opus 5.5, 2026-10-11	`pattern_evaluation.csv`, `Pattern_count.csv`, 9 UML diagrams	**Correct with stated limits.** All 117 final-version locations checked to exist; role mapping and status words need human review.
Part 3	Claude Opus 5.5, 2026-10-10	Factory Method spec, C++ port and measurement script	**Partially correct.** C++ 7/7 and LOC 71 confirmed. Correction: paths in the CSV named `Part3/FactoryMethod_VehicleCategoryRules/`; the folder is `Part3/Factory_Method/`.
Part 3	Claude Opus 5.5, 2026-10-10	Observer ports in four languages	**Correct.** All four re-run, 8/8; LOC matches `measure.py`. Ports are deliberately simplified (synchronous, in-memory).
Part 3	Claude Opus 5.5, 2026-10-10	DI ports in four languages	**Partially correct.** 10/10 in the author's layout after fixing the Java overload bug; not re-run in this repository.

Inputs and outputs: Part 1: prompt `Part1/Part1_prompt.txt`, input `Part1/llm/input/part1_source_context.txt` (baseline `dfc61ab8`), output `Part1/llm/output/pattern_discovery_raw.txt`, prompt history `Part1/llm/Part1_LLM_Prompt_History.pdf`. Part 2: prompts `Part2/part2_prompt.txt` and `Part2/changes/Change4/prompt.md`; inputs are the baseline source files and change documents listed in `Part2/llm.csv`. Part 3: prompt `Part3/llm/part3_prompt.txt`; inputs are the C# sources of the selected instances.

Claims and evaluation: See the table above. Reviewer: Team B6. Evidence files are named in the `verification_file` column of each `llm.csv`.

Generated code: Used: the CH04 implementation, the CH01 reconstruction and integration edits in `Part2/code`, and the Part 3 Observer ports, Factory Method C++ port and measurement scripts. All were compiled and tested: `dotnet test` for Part 2 (297/297) and the shared test runners for Part 3.

6 Findings, contributions and references

Findings:

1. The original code is shaped by the framework: its nine confirmed instances are enterprise, architectural, concurrency and integration patterns. It contains no application-authored behavioural GoF pattern.
2. The patterns that were absent or doubtful at the baseline (State, Factory, Strategy, Observer) were exactly the extension points the new requirements needed; each change introduced one, taking the count from 9 to 13 types and instances.
3. Existing collaborations absorbed the changes: no instance was removed or replaced; DI was extended by every change and the Generic Repository gained clients without edits.
4. Refactoring was applied where a real smell existed: static pricing coupling with a hidden clock (CH03) and a long mixed-responsibility method (CH04). The other two changes were purely additive.
5. Behaviour was preserved: the original tests keep their results, the suite grew from 183 to 297 tests, and the only baseline failure was a clock dependency removed by injection.
6. In Part 3 the participant structure is the same in Java, Python, JavaScript and C++; the type system changes the LOC, the complexity and when mistakes are caught, not the pattern.

Limitations. (a) CH01's original source is not in the repository; the CH01 code in `Part2/code` is a reconstruction from its documents and test names. (b) The evolved application was built and tested but not run against MySQL. (c) The CH02 and CH04 commit hashes exist only in local clones, and CH01 and CH03 have no after-commit hash. (d) CH03's own test log is a transcription. (e) No complexity, coverage or coupling metrics exist for the C# application. (f) Part 3 timings include process start-up and come from different machines. (g) Some LLM records reference files not present in the repository (`Part1/llm/human_verification.txt`, `Part2/llm_output_ch04.md`, `Part3/llm/part3_observer_prompt.txt`).

Contributions:

Roll number	Name	Batch	Contribution and evidence
AM.SC.U4CSE23251	Samridhi Singh	CSE C (2023–2027)	CH01 fleet-maintenance scheduling (State), 50 tests: `Part2/changes/Change1`
AM.SC.U4CSE23211	Bhavana P H	CSE C (2023–2027)	CH02 vehicle categories (Factory Method), 24 tests, run-time evidence on MySQL: `Part2/changes/Change2`
AM.SC.U4CSE23238	Neema Vinod	CSE C (2023–2027)	CH03 pricing policies (Strategy) and refactoring, 22 tests: `Part2/changes/Change3`
AM.SC.U4CSE23266	Vani Sugovind S R	CSE C (2023–2027)	CH04 booking notifications (Observer) and refactoring, 18 tests: `Part2/changes/Change4`

Part 1, Part 3, the integration and this report were team work.

References:

1. Naveen-verma-7697, *FleetManagment* (WanderCar / Fleman), GitHub, commit `dfc61ab8`. `https://github.com/Naveen-verma-7697/FleetManagment`
2. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
3. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (Repository, Service Layer, Data Transfer Object, Data Mapper, Unit of Work).
4. M. Fowler, "Inversion of Control Containers and the Dependency Injection pattern", 2004. `https://martinfowler.com/articles/injection.html`
5. M. Fowler, "Strangler Fig Application". `https://martinfowler.com/bliki/StranglerFigApplication.html`
6. M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2018 (Extract Class, Move Method, Extract Interface).
7. Microsoft Learn: dependency injection in ASP.NET Core; ASP.NET Core middleware; background tasks with hosted services; `System.Threading.Channels`; `TimeProvider`.
8. B. Stroustrup, H. Sutter (eds.), *C++ Core Guidelines* (ownership with `unique_ptr`/`shared_ptr`).
9. Tools: .NET SDK 8.0.425, NUnit 3.14, Moq 4.20, FluentAssertions 6.12, EF Core 8, PlantUML 1.2024.8, Graphviz, `lizard` 1.24.1, OpenJDK 17/24/26, CPython 3.13/3.14, Node.js 22/24, g++ 15.2, clang 21.1 (zig 0.16).

7 Team folder and datasets

Repository: `https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy`, folder `B6`.

```
B6/
  README.md                          team members
  B6_Report.tex, B6_Report.pdf       this report
  data/
    code/WanderCar_DotNet/           original application source (dfc61ab8)
    test_result/                     baseline build/test logs, trx, screenshots
  Part1/
    part1_patterns.csv, part1_pattern_count.csv
    inspected_modules.txt, baseline_commit.txt
    pattern_architecture/            6 diagrams (.puml, .dot, .png)
    Part1_prompt.txt, llm.csv, llm/  (input, raw output, prompt history)
  Part2/
    code/WanderCar_DotNet/           evolved application (CH01-CH04)
    code/test_result.txt             297/297 log
    changes/Change1 ... Change4/     requirement, diff, pattern effects,
                                     refactoring, metrics, test results
    changes/integration/             one patch per step + README
    pattern_evaluation.csv, Pattern_count.csv
    diagrams/                        9 updated UML diagrams
    part2_prompt.txt, llm.csv
  Part3/
    Factory_Method/, Observer_Pattern/, Dependency_Injection_Pattern/
                                     Java, Python, JavaScript, C++ + spec
    strategy_pattern/, repository_pattern/
    llm/  part3_prompt.txt, llm.csv, pattern_crosslanguage.csv
```

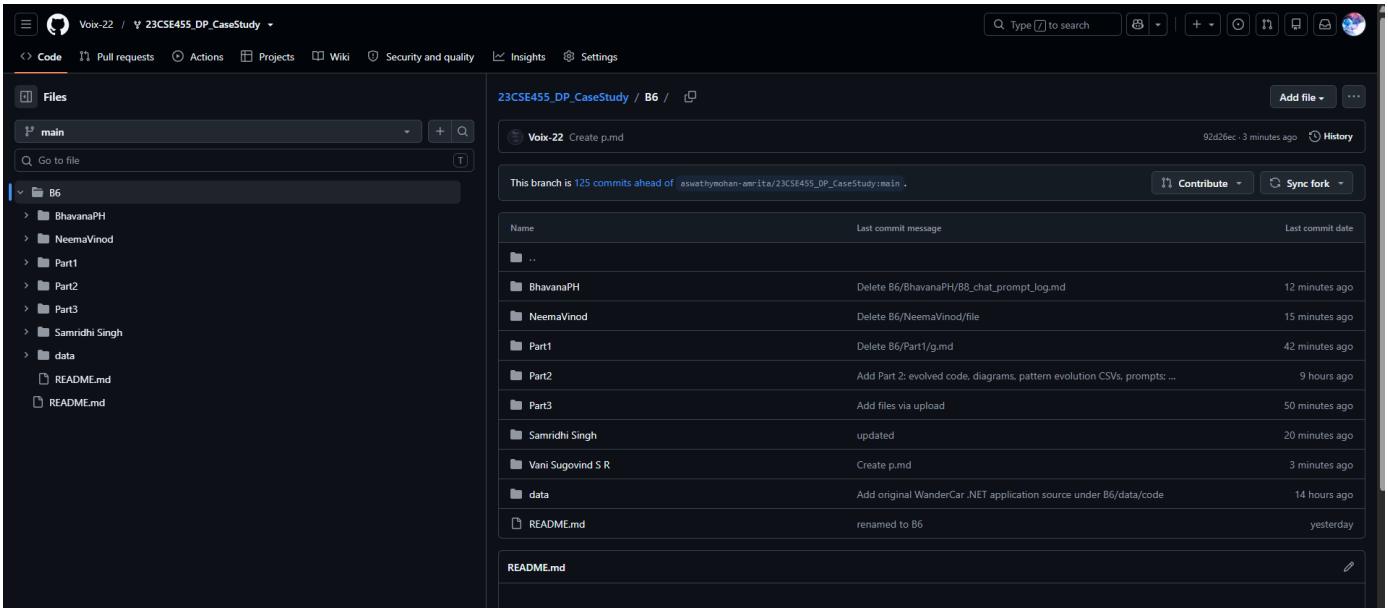


Figure 11: B6 Repository